Import Statements

In [11]:
```
import custom_cmap
import os
import sys
from PIL import Image
from IPython.display import display
import matplotlib.pyplot as plt
import matplotlib as mpl
import pandas as pd
from moria import reduce

from pathlib import Path
from astropy.visualization import PercentileInterval

from astropy.io import fits
from astropy.visualization import LogStretch, ImageNormalize
import plotly.express as px
import numpy as np
from astropy.visualization import ImageNormalize, AsinhStretch, SqrtStretch, LogStretch, PowerStretch

plt.rcParams.update({'font.size':25})
plt.rc('text', usetex=True)
%config InlineBackend.figure_format = 'retina'
%load_ext autoreload
%autoreload 2
import time as clock
```

Understanding the main directory.

The data directory should be organized as follows. You can look at the sample data folder to understand the directory strucutre you need.

00.DATA

01.XYM

02.CMD
03.LOC_TRANS
04.PSF_EXTRACT
05.COORD_TRANS(OPTIONAL)
06.FIT
07.CALIBRATION
All the necessary scripts are included in the sample data directory. The simplest way to run MORIA on your target is to copy the entirety of the "data" folder here, put your exposures in "data/00.DATA", then beginning with the demo.

Using the bright nearby reference stars identified in the CMD, more local transformations are derived for each exposure. Pixel data surrounding the target star and the selected PSF-star candidates are extracted from each exposure and accurately transformed into a reference frame corrected for distortion. The extracted pixel list contains the flux location of each pixel relative to the star centers and is fundamental for building the PSF model.

For each filter, using the stars selected to have similar magnitudes and colors to the target star, MORIA builds a PSF model. Candidate PSF stars are evaluated using residual images from PSF subtraction and stars that show significant residuals due to blending or large Poisson noise are excluded in a second iteration used for PSF modeling. The final output is to derive a high-resolution PSF model for each filter.

NOTE: chmod +x program.src is a useful command to use whenever permissions are denied for a script.

In [12]:
```
directory = os.getcwd()
#code_directory = os.path.abspath("..")
#sys.path.append(code_directory)
```

Step 0

We assume you ran output_stacks.ipynb and then cmd_diagram.ipynb (in that order)

Step 1

Generate a more local transformation from each frame into the reference frame, then use this transformation to extract the pixels from each exposure and accurately transform their locations into the reference frame so that we can use them to solve for a PSF and then use this PSF to model the target star. The step below will take quite long to run.

In [26]:
```
start = clock.time()
reduce.loc_trans(directory)
end = clock.time()
print(end-start)
```

Step 2

Generate a local PSF for each filter, using the stars that are similar in magnitude and color to the target star. The magnitude similarity is important because the CTE losses are expected to make PSF shapes magnitude dependent. The number you enter here should be the number of stars you found in the file "NEARBY_SIM_STARS.XYIVB_targ" in 02.CMD.

In [20]:
```
start = clock.time()
reduce.extract_psf_1(directory)
end = clock.time()
print(end-start)
```

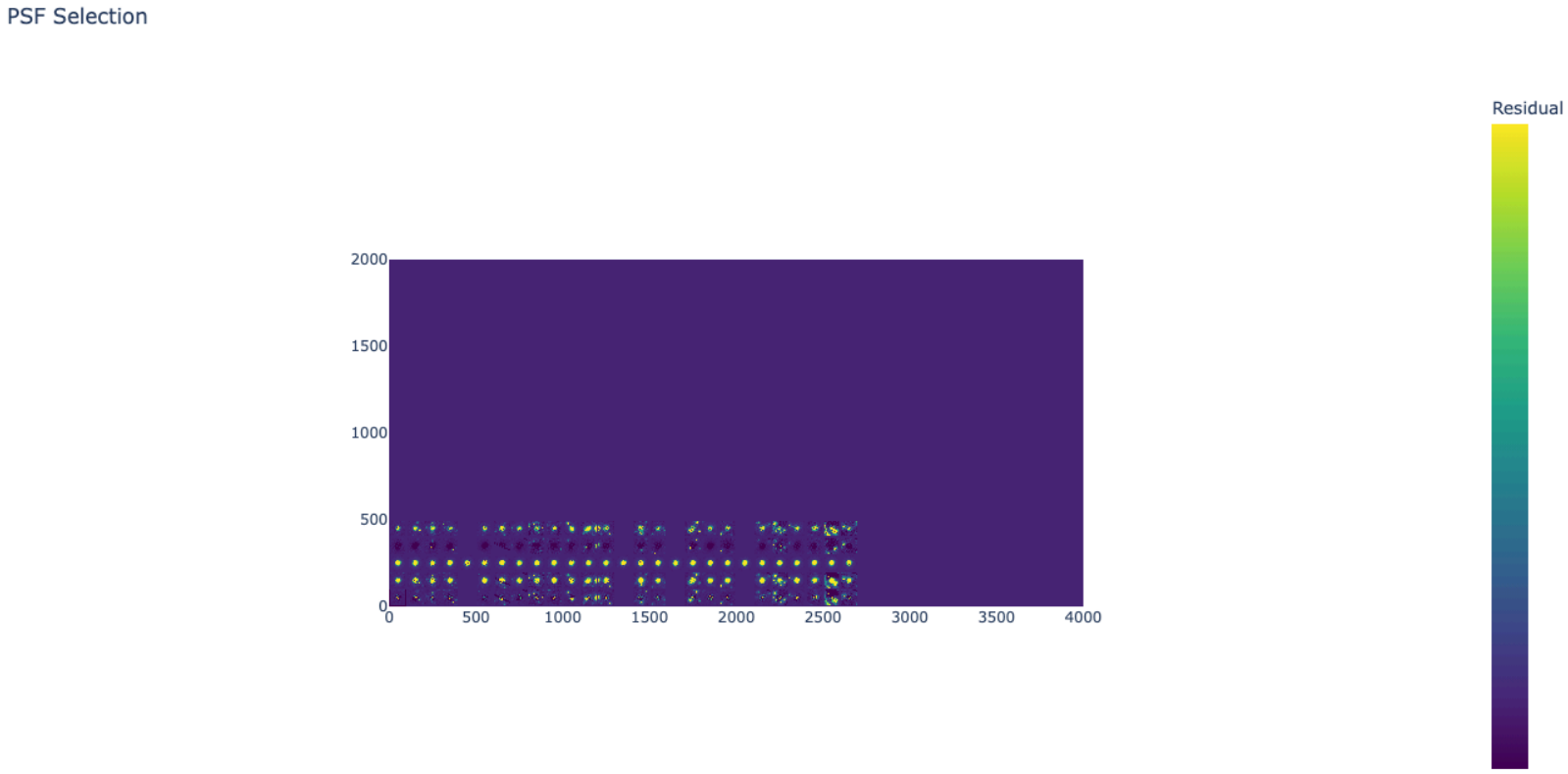
Step 3

Loook at the fit file generated above. The fits file are stored as "show_str_simst1.fits" in 04.EXTRACT_PSF/F814W for the F814W filter and 04.EXTRACT_PSF/F606W for the F606W filter.

You are only concerned by the the bottom panel. Go through it to select "good" stars. Ensure that the residuals are smooth

In [21]:
```
fit_file = Path(directory).resolve()/f"04.EXTRACT_PSF/F814W/show_str_simst1.fits"

hdul = fits.open(fit_file)
data = hdul[-1].data
hdul.close()
interval = PercentileInterval(99)
scaled = interval(data)
```

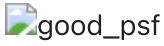
```
fig = px.imshow(scaled, origin='lower', color_continuous_scale='viridis', title="PSF Selection", aspect='e

fig.update_layout(width=750, height=750, coloraxis_colorbar=dict(title="Residual", tickvals=[]))
fig.show()
```



Here are some examples of bad PSF panels. In these images you can see either black subtractions near the center of the image or residuals that are not "smooth."

bad_psf bad_psf

You want to select panels that have a smooth residual Here is an example of a good PSF panels



Step 4: Input the PSF stars you like from your selection above

Input the panel number for the PSF stars you like as a list below. The first panel is panel 0:

In [22]:
```
good_panels = [1, 3, 7, 9, 15, 18, 19]
```

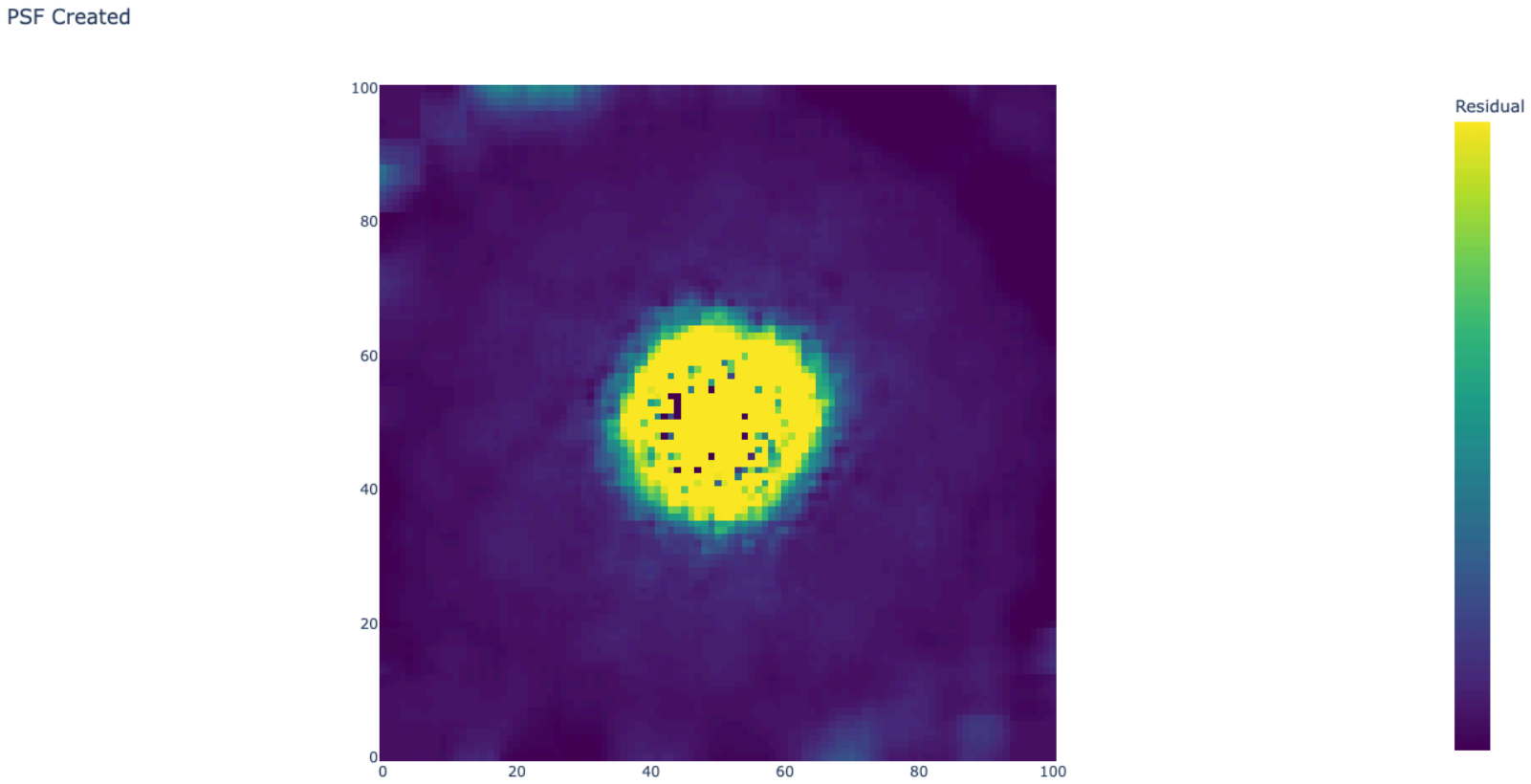
In [23]:
```
#Change number_of_sim_stars to the number of sim stars you have
number_of_sim_stars=28
good_psf = np.zeros(number_of_sim_stars, dtype = int)
for index in good_panels:
    good_psf[index] = 1
```

Step 5: Now run PSF Extract again.

This will create a good PSF for your target.

In [24]:
```
reduce.extract_psf_2(good_psf, directory)
```

In [25]:
```
fit_file = Path(directory).resolve()/f"04.EXTRACT_PSF/F814W/psfout_simst.fits"

hdul = fits.open(fit_file)
data = hdul[-1].data
hdul.close()
interval = PercentileInterval(90)
scaled = interval(data)

fig = px.imshow(scaled, origin='lower', color_continuous_scale='viridis', title="PSF Created", aspect='equa

fig.update_layout(width=750, height=750, coloraxis_colorbar=dict(title="Residual", tickvals=[]))
fig.show()
```



In [ ]: